

Validación de datos

¿Porque validar datos?

Cuando trabajamos con datos ingresados por un usuario, leídos desde un archivo o extraídos de un string, no podemos asumir que siempre tendrán el formato esperado.

Validar los datos nos permite revisar mínimamente su contenido antes de usarlos.

Por ejemplo:

- Si esperamos un número y recibimos texto, el programa puede fallar al convertirlo.
- Si esperamos datos separados por “,” y la coma no está, no podremos separar correctamente.
- Si esperamos un nombre y recibimos un número, quizás el programa no falle, pero el dato será inconsistente.

Validar ayuda a construir programas más seguros, claros y confiables.

Las herramientas de validación son muchas, estás son algunas para tener ideas disparadoras.

Validando números en la entrada

La función `isnumeric` no permite determinar si un texto es o no un número:

```
"1996".isnumeric()
# True
"Mil novecientos noventa y nueve".isnumeric()
# False
```

NOTA: Python no ofrece ningún mecanismo para validar floats sobre strings.

A partir de esto con un simple `if` podemos resguardar nuestro programa:

```
fecha_nacimiento = 0
fecha_nacimiento_raw = input("Ingrese su fecha de nacimiento [yyyy]: ")
if fecha_nacimiento_raw.isnumeric():
    fecha_nacimiento = int(fecha_nacimiento_raw)
    print("Fecha de nacimiento asignada exitosamente")
else:
    print("Fecha de nacimiento invalida")
    exit() # Usamos exit para salir inmediatamente dle programa
```

Validando tamaño de la entrada

En ocasiones no va a interesar que el tamaño de una entrada sea el que necesitamos, por ejemplo, para el caso del año, si lo vamos a utilizar matemáticamente, estamos obligados a tener los 4 caracteres.

La opción más simple para esto es len con un comparador.

```
len("1996") == 4 # True
len(" 1996") == 4 # ¿ Que pasa acá?
```

A partir de esto con un simple if podemos resguardar nuestro programa:

```
fecha_nacimiento = 0
fecha_nacimiento_raw = input("Ingrese su fecha de nacimiento [yyyy]: ")
if fecha_nacimiento_raw.isnumeric() and len(fecha_nacimiento_raw.strip())
== 4:
    fecha_nacimiento = int(fecha_nacimiento_raw)
    print("Fecha de nacimiento asignada exitosamente")
else:
    print("Fecha de nacimiento invalida, se esperaban 4 digitos, EJ:
1992")
    exit() # Usamos exit para salir inmediatamente del programa
```

Validando ingreso de palabras

Python provee `isalpha` para validar el ingreso de palabras, no soporta texto con espacios por ejemplo, tampoco caracteres especiales ni números.

```
"Maria".isalpha() # True  
"Maria Cecilia".isalpha() # False
```

Nos es muy útil para pedir nombres o cuando necesitamos respuesta de Si y No.

NOTA: también existe `isalnum()` valida que un texto sea alfanumérico, es decir caracteres de A-Z y números sin espacios ni símbolos extraños.

```
primer_nombre = ""  
primer_apellido = ""  
primer_nombre_raw = input("Ingrese su primer nombre: ")  
primer_apellido_raw = input("Ingrese su primer apellido: ")  
if primer_nombre_raw.strip().isalpha() and  
primer_apellido_raw.strip().isalpha():  
    primer_nombre = primer_nombre_raw  
    primer_apellido = primer_apellido_raw  
    print("Nombre y apellido registrado correctamente")  
else:  
    print("Error nombre o apellido invalido")
```

Validando caracteres esperados

Cuando esperamos el texto en cierto formato, o por ejemplo separado por “,” podemos utilizar funciones como count o find.

count cuenta la cantidad de apariciones, para datos estructurados donde sabemos cuántos columnas esperamos nos viene muy bien.

```
"Pedro,Martinez,19,2004".count(",") == 3 # True
```

```
nombre_apellido = input("Ingrese nombre,apellido: ")
if nombre_apellido.count(",") == 1:
    nombre, apellido = nombre_apellido.split(",")
    print("Validación correcta")
else:
    print("Error se debe ingresar nombre,apellido")
```

Ejercicios

Ejercicio 1 - Cargar nota de parcial

Pedir al usuario una nota de parcial.

Validar que:

- Sea numérica.
- Esté entre 0 y 10.

Si es válida, mostrar

Ejercicio 2 - Lista de productos separada por coma

Pedir al usuario que ingrese productos separados por coma.

Validar que se ingresó más de dos productos, y que ninguno quedó vacío.

Ejercicio 3 - Validar patente simple

Pedir al usuario una patente con el siguiente formato:
Dos letra + tres números + dos letras (AB123CD)

Validar que:

- No tenga espacios.
- Tenga 7 caracteres.
- Contenga solo letras y números.
- El formato

Ejercicio 4 - Validar float

Pedir al usuario un número flotante implementar un mecanismo para validar que el dato ingresado es correcto.